National University of Sciences and Technology

School of Mechanical and Manufacturing Engineering

RAI-833 Deep Learning

Tutorial 3: ANN Classifier Report

Submitted to:

Dr. Shahbaz Khan

Submitted by:

Hamza Suhaib Qarni

538866

SMME

April 11, 2026

Abstract

This report covers the implementation, optimization, and evaluation of an Artificial Neural Network (ANN) for handwritten digit classification utilizing the MNIST dataset. The objective of this assignment was to implement a baseline model in a PyTorch framework and tune its hyperparameters.

Key takeaways:

- **Architecture:** A deeper network configuration (256 → 128 → 64) slightly outperformed the base ReLU and Tanh models, achieving a 97.76% validation accuracy.
- **Optimizers:** The Adam optimizer performed the best, yielding the highest accuracy (97.73%) compared to RMSprop (97.42%) and SGD (94.53%).
- **Regularization:** Applying 20% Dropout, L2 Regularization, and Early Stopping successfully mitigated overfitting, automatically halting training at Epoch 11 and yielding a final test accuracy of 97.59%.

Contents

1 Introduction **2**

2 Base PyTorch Implementation **2**

3 Architecture Analysis **2**

4 Optimizer Analysis **3**

5 Regularization & Early Stopping **4**

5.1 Classification Report 5

6 Error Analysis **6**

7 Conclusion **7**

1 Introduction

The MNIST dataset serves as a standard benchmark for computer vision classification tasks, consisting of 28x28 pixel grayscale images of handwritten digits (0-9). The goal of this assignment is to construct a fully connected feed-forward Neural Network capable of classifying these images.

A major component of this tutorial involved migrating the provided Keras codebase into PyTorch. Key structural differences observed during this transition included PyTorch's native handling of image normalization via `transforms.ToTensor()` and its internal combination of Softmax and negative log-likelihood within the `nn.CrossEntropyLoss` function.

2 Base PyTorch Implementation

The baseline model was constructed with an input layer flattening the 28×28 image into 784 features, followed by two hidden layers of 128 and 64 neurons utilizing ReLU activations. The output layer consisted of 10 neurons corresponding to the digit classes. The initial model was trained using the Adam optimizer for 10 epochs. This baseline achieved an initial test accuracy of **0.9758**.

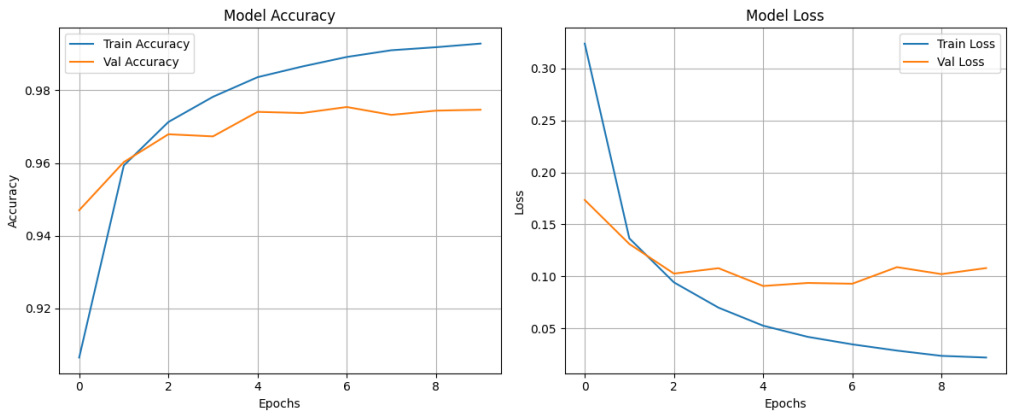

Figure 1: Training and Validation curves for the Base PyTorch Model.

3 Architecture Analysis

To understand the impact of model capacity and non-linearity, two variations of the base model were experimentally tested:

- **Tanh Model:** The ReLU activations were replaced with Hyperbolic Tangent (`Tanh`) functions.
- **Deeper Model:** The network capacity was increased to three hidden layers ($256 \rightarrow 128 \rightarrow 64$).

As shown in Table 1, the Deeper architecture provided the highest validation capacity, while the Tanh model marginally underperformed compared to standard ReLU.

Table 1: Performance Comparison of Network Architectures (Adam Optimizer)

Architecture	**Validation Accuracy**
Base (ReLU: 128 → 64)	0.9766
Tanh (128 → 64)	0.9758
Deeper (256 → 128 → 64)	**0.9776**

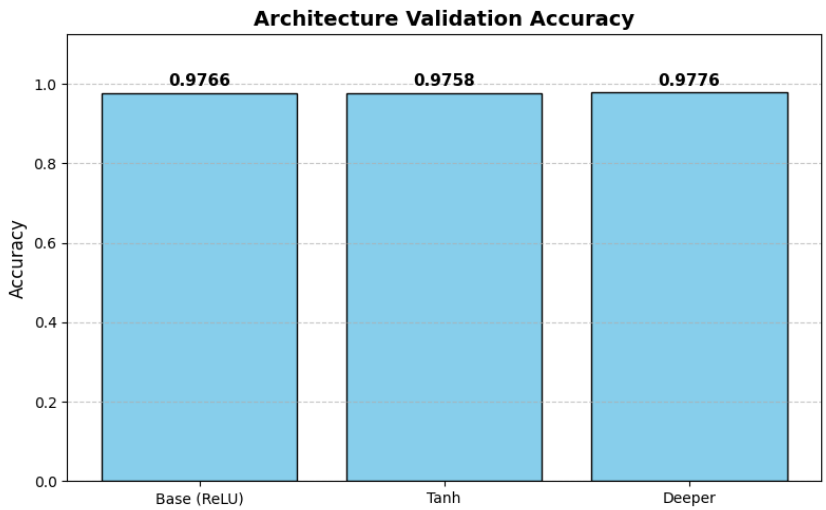

Figure 2: Validation Accuracies across different architectures.

4 Optimizer Analysis

Using the baseline ReLU architecture, the model was trained across three distinct optimization algorithms to evaluate convergence speed and final accuracy.

- **SGD (Stochastic Gradient Descent):** Took consistent steps but required a higher learning rate and converged slower (94.53% accuracy).
- **RMSprop & Adam:** Both adaptive learning rate methods converged significantly faster. Adam achieved the highest overall accuracy (97.73%) by combining momentum and RMS scaling, while taking a similar total runtime to RMSprop.

Table 2: Performance Comparison Across Optimizers

Optimizer	**Learning Rate**	**Validation Accuracy**	**Training Time (s)**
SGD	0.010	0.9453	**74.34**
RMSprop	0.001	0.9742	76.93
Adam	0.001	**0.9773**	77.96

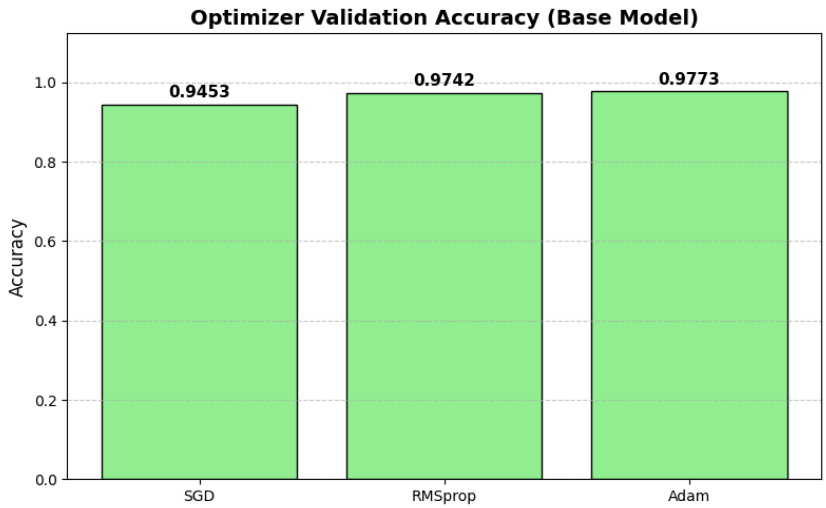

Figure 3: Validation Accuracies across different optimizers.

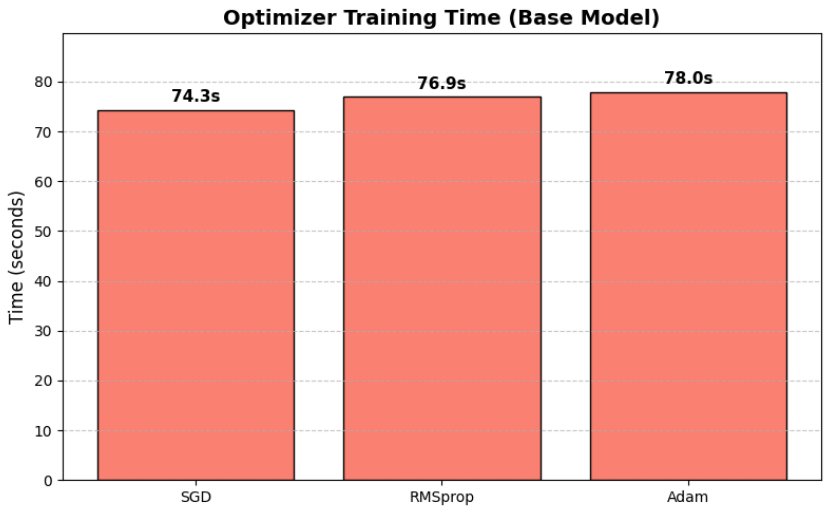

Figure 4: Training times (in seconds) for each optimizer.

5 Regularization & Early Stopping

Standard training without regularization eventually causes the model to overfit, where the training loss drops but validation loss plateaus. To address this, a regularized model was developed featuring:

1. **Dropout Layers ($p = 0.2$):** Randomly dropping 20% of neurons during training.
2. **L2 Regularization (`weight_decay=1e-4`):** Penalizing excessively large weights.
3. **Early Stopping (`patience=3`):** Halting training if performance degraded for 3 consecutive epochs.

The Early Stopping mechanism successfully triggered, halting training at **Epoch 11** and restoring the optimal weights from **Epoch 8**. The final regularized test accuracy was **0.9759**.

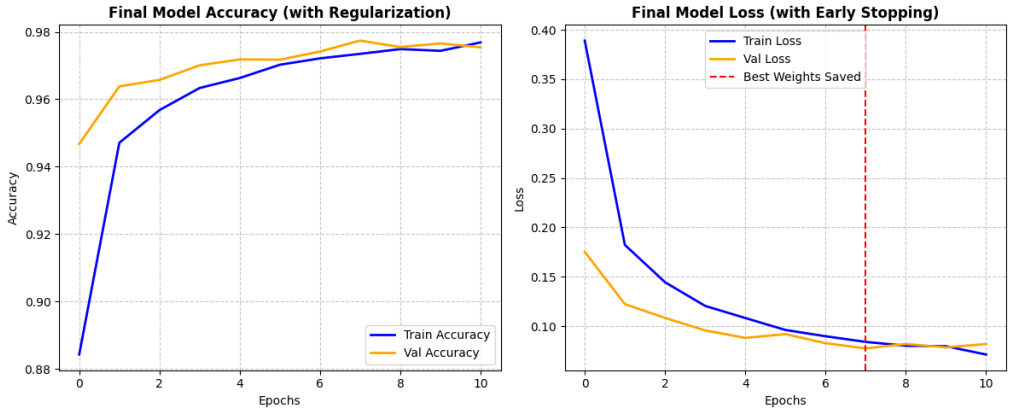

Figure 5: Learning curves of the regularized model showing the Early Stopping trigger.

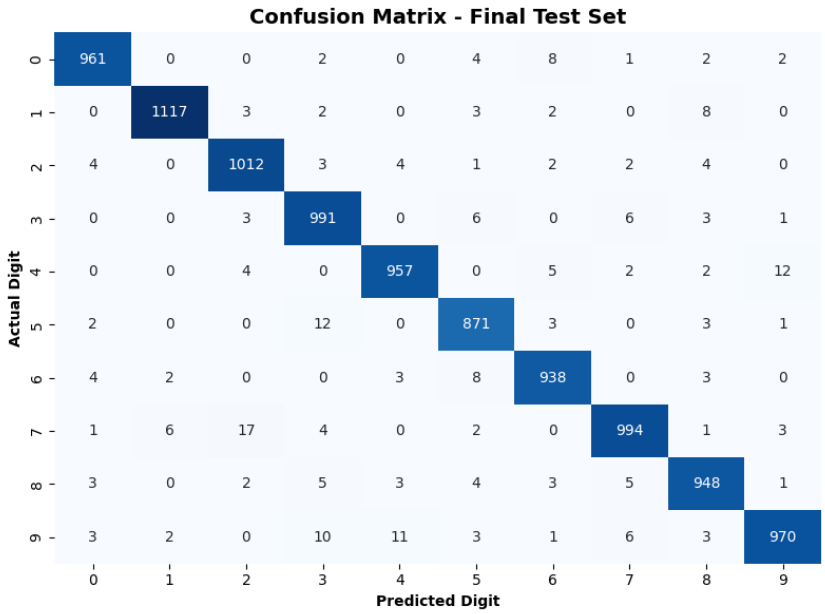

Figure 6: Confusion Matrix evaluated on the final unseen test set.

5.1 Classification Report

The precision, recall, and F1-scores across all ten digit classes confirm that the regularized model is highly balanced.

Table 3: Scikit-Learn Classification Report (Final Model)

Digit Class	Precision	Recall	F1-Score	Support
0	0.9826	0.9806	0.9816	980
1	0.9911	0.9841	0.9876	1135
2	0.9721	0.9806	0.9764	1032
3	0.9631	0.9812	0.9720	1010
4	0.9785	0.9745	0.9765	982
5	0.9656	0.9765	0.9710	892
6	0.9751	0.9791	0.9771	958
7	0.9783	0.9669	0.9726	1028
8	0.9703	0.9733	0.9718	974
9	0.9798	0.9613	0.9705	1009
Accuracy			**0.9759**	**10000**
Macro Avg	0.9757	0.9758	0.9757	10000
Weighted Avg	0.9760	0.9759	0.9759	10000

6 Error Analysis

To interpret the model's performance, samples from the test set were plotted alongside their predicted and true labels.

Figure 7: A random grid of successfully classified handwritten digits.

Although the model achieved high accuracy, analyzing the misclassified images helps identify its limitations. As shown below, many of the misclassified digits contain severe

human handwriting artifacts, such as overlapping loops or incomplete strokes. These visual ambiguities confuse the network similarly to how a human reader might misinterpret poorly written text.

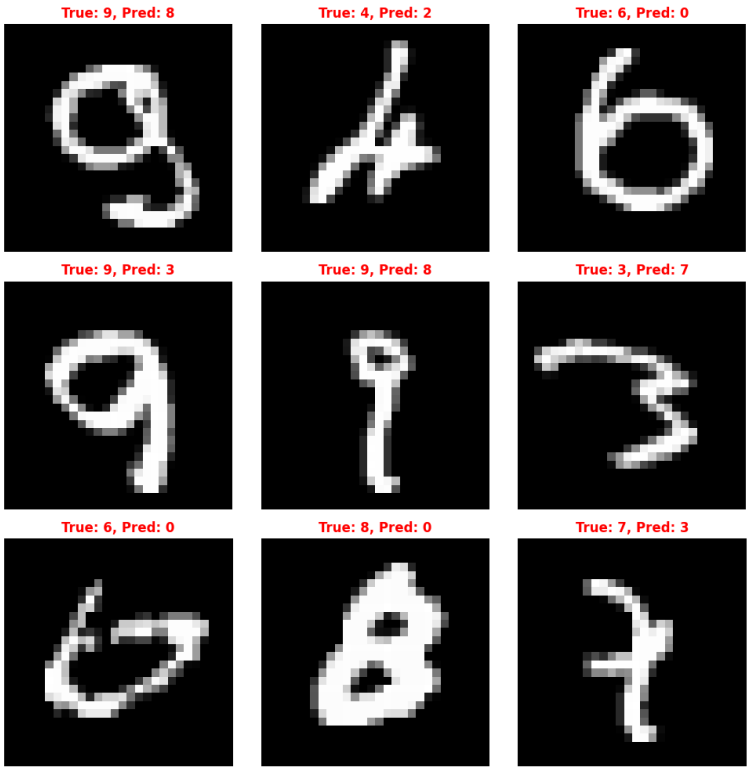

Figure 8: Grid of misclassified digits highlighting ambiguous handwriting.

7 Conclusion

This assignment successfully demonstrated the end-to-end pipeline of building and optimizing an Artificial Neural Network in PyTorch. The experiments proved that while increasing network depth and utilizing adaptive optimizers like Adam boosts baseline performance, it inherently accelerates overfitting on the training set.

Implementing regularization techniques like Dropout, L2 Regularization, and Early Stopping was highly effective in preventing overfitting and creating a robust model that generalizes well to unseen data.